

EchoSphere v2.2: Master Technical Specification & Software Architecture Manual

Exhaustive Engineering Reference of Front-End Client, Asynchronous Core Backend & AIML Microservices Engine

Document Specification: Complete System SRS & Software Architecture Manual
Authors: EchoSphere Systems Engineering & AI Research Division | **Version:** 2.2.0 | **Target OS:** Android, Windows Desktop, Linux, iOS
Core Stack: Flutter 3.x / Dart 3.x, Python 3.11 FastAPI, PostgreSQL 15, SQLAlchemy 2.0 Async, Google Gemini 1.5/3.6, WebSockets, FCM, Soundpool

1. Executive Abstract & System Vision

EXECUTIVE ABSTRACT & SYSTEM SPECIFICATION

Abstract — Higher education institutions present a complex operational environment where information dissemination must balance speed, strict institutional governance, departmental isolation, and real-time urgency. Traditional notice boards are physically tethered and prone to visual noise. Informal chat platforms (e.g., WhatsApp, Telegram) lack verifiable authorization, exposing institutions to misinformation and uncontrolled notice sprawl. Conversely, traditional enterprise ERP platforms are bloated, lacking specialized real-time announcement engines and modern artificial intelligence integration.

EchoSphere v2.2 resolves these challenges by introducing a smart, multi-tiered campus announcement management ecosystem. Built upon a decoupled 3-tier microservices paradigm featuring a cross-platform Flutter frontend client, an asynchronous FastAPI backend core, and a dedicated AI/ML service engine, EchoSphere delivers an end-to-end workflow for notice creation, approval, priority escalation, and multi-channel distribution. The application enforces a strict 6-tier Role-Based Access Control (RBAC) hierarchy (Student, Teacher, HoD, College Administrator, Principal, Developer Admin), integrating real-time WebSockets, automated push notifications, audio speaker broadcast queues, immutable audit trails, and domain-bounded LLM assistance.

1.1 Core Engineering Objectives

- 1. Zero Communication Delay:** Instantaneous real-time notice delivery across mobile and desktop platforms via WebSockets and Firebase Cloud Messaging (FCM).
- 2. Guaranteed Zero Layout Overflow:** Strict technical enforcement of responsive Flutter layout constraints (`Expanded`, `Flexible`, `SingleChildScrollView`, `Wrap`) ensuring zero pixel rendering overflow errors on all screen sizes from 320px compact mobile screens to 4K desktop monitors.
- 3. Strict Hierarchical Governance:** Multi-stage approval routing preventing unverified notices from reaching students without explicit HoD, Administrator, or Principal authorization.
- 4. AI-Powered Quality & Assistance:** Integrated natural language processing for notice drafting expansion, executive single-sentence summarization, priority recommendation, spam/misinformation filtering, duplicate detection, and domain-bounded student RAG Q&A.
- 5. Immutable Security & Auditability:** Complete non-repudiable audit trailing capturing all credential authentications, announcement creation/modification events, approval reviews, and system configuration updates.

1.2 UI Consistency Across User Roles Rule

Workspace Rule Enforcement: UI/UX improvements, design styling enhancements, and layout polish applied to the Developer Administrator (devadmin) pages must automatically propagate to the pages of all other roles (Student, Teacher, Head of Department, College Admin, Principal). This maintains a 100% uniform visual experience while strictly enforcing role capabilities and authorization boundaries at the controller level.

2. Microservices Architecture & Design Paradigm

EchoSphere is structured around a decoupled microservices architecture designed for high availability, independent scaling, and isolation of concerns. The platform separates client presentation, core business logic, relational persistence, and artificial intelligence operations into distinct service layers.

Tier Layer	Component & Tech Stack	Port & Protocol	Core Responsibilities & Internal Modules
Presentation Client	Flutter 3.x, Dart 3.x, Provider, GetX, Soundpool	Native OS App (Android / Win / iOS)	Role-adaptive UI dashboards, responsive layouts, offline cache, sound effects player, real-time WebSocket subscriber, FCM listener.
Core Application API	Python FastAPI 0.100+, Uvicorn, SQLAlchemy 2.0 Async, Alembic	Port `8000` (REST HTTP + WS)	OAuth2/JWT Auth, User Management, Announcement State Machine, Approval Queues, Audit Logging, Real-time WebSocket Router.
AIML Intelligence Engine	FastAPI, Google Gemini 1.5/3.6, Ollama, PyTorch, HuggingFace NLP	Port `8001` (REST HTTP)	Notice drafting & expansion, executive summarization, spam/misinformation classifier, duplicate detector, student RAG Q&A; engine.
Database Storage	PostgreSQL 15 Alpine, Redis Cache, Docker Volumes	Port `5432` (PostgreSQL Protocol)	ACID transactional persistence, multi-role user schemas, relational integrity, index optimization on USN and Employee IDs.

2.1 Inter-Service REST & WebSocket Protocols

1. **Client Requests:** The Flutter frontend dispatches REST requests over HTTPS to the Core Backend (`http://localhost:8000/api/v1`).
2. **Authentication & Authorization:** The Core Backend validates incoming JWT Bearer tokens against the secret key and evaluates user role permissions via FastAPI dependency guards (`require_roles`).
3. **AI Task Delegation:** When a user requests notice expansion, summarization, or AI Q&A;, the Core Backend forwards the payload to the AIML service (`http://localhost:8001/ai/...`).
4. **Real-Time Event Dispatch:** Upon announcement approval or emergency escalation, the Core Backend broadcasts WebSocket frames to active connected clients and triggers FCM push notifications to offline mobile devices.

3. Hierarchical Multi-Role Access Control (RBAC) & Security

EchoSphere enforces a strict 6-level role hierarchy. Authorization increases progressively from top to bottom. Each role inherits permissions from lower roles unless explicitly restricted by institutional policy.

3.1 Role Hierarchy & Inheritance Model

Role Hierarchy Matrix:

[1] Student (Baseline Read-Only & Student AI Q&A;)

■■■■ [2] Teacher (Departmental Notice Creation & Student Account Management)

■■■■ [3] Head of Department (HoD) (Department Approval Authority & Teacher Management)

■■■■ [4] College Administrator (Institutional Admin & Emergency Override)

■■■■ [5] Principal (Full Academic Authority & Global Approval)

■■■■ [6] Developer Administrator (Superuser Governance & System Config)

3.2 Differentiated Authentication Flow Matrix

Role Name	Primary Credential	Secondary Verification	Format & Regex Rule	Allowed Capabilities
Student	University Seat No. (USN)	Account Password	<code>^[0-9][A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{3}\$</code> (e.g. <code>1EC22CS001`</code>)	Read announcements, search feed, receive push notifications, interact with AI Q&A.;
Teacher	Official Email	Employee ID	Prefix <code>`EMP`</code> or <code>`TCH`</code> (e.g. <code>`TCH001`</code> , <code>`DBITAIMLT022022`</code>)	Create notices, request approval, manage student accounts in department.
Head of Dept (HoD)	Official Email	Employee ID	Prefix <code>`HOD`</code> (e.g. <code>`HOD001`</code>)	Approve/reject teacher notices, edit department announcements, create teacher/student accounts.
College Admin	Username / Email	Employee Security ID	Prefix <code>`ADM`</code> (e.g. <code>`DBITADM001`</code> , <code>`ADM001`</code>)	College-wide announcements, emergency broadcasts, full user account management.
Principal	Executive Email	Master Password	Official Email Domain (e.g. <code>`PRI001`</code>)	Global approval authority, edit/delete any notice, reschedule institutional broadcasts.
Developer Admin	Superuser Username	System Secret Key	Username <code>`devadmin`</code> (e.g. <code>`DEVADM01`</code>)	Full database seeders, system configuration, global UI design cascade control.

3.3 Exhaustive 18-Feature Permission Grid

Permission / Feature Area	Student	Teacher	HoD	Admin	Principal	DevAdmin
View Notice Feed & Search	YES	YES	YES	YES	YES	YES
Ask AI Student Assistant Q&A;	YES	YES	YES	YES	YES	YES
Create Department Draft Notice	NO	YES	YES	YES	YES	YES
Request Approval for Notice	NO	YES	YES	YES	YES	YES
Approve Department Notices	NO	NO	YES	YES	YES	YES
Reject Notice with Feedback	NO	NO	YES	YES	YES	YES
Create Emergency Alert	NO	NO	NO	YES	YES	YES
Broadcast College-Wide	NO	NO	NO	YES	YES	YES
Manage Student Accounts	NO	YES	YES	YES	YES	YES
Manage Teacher Accounts	NO	NO	YES	YES	YES	YES
Manage Admin Accounts	NO	NO	NO	NO	YES	YES
Manage Speaker Broadcast Queue	NO	NO	YES	YES	YES	YES
View System Audit Logs	NO	NO	NO	YES	YES	YES
Trigger Seeders & Config	NO	NO	NO	NO	NO	YES

4. Core Announcement Engine & Workflow Dynamics

The Announcement Engine manages the complete lifecycle of institutional notices, enforcing state transitions, approval queue routing, priority escalation, and multi-channel notification dispatching.

4.1 Announcement Lifecycle State Machine

1. **DRAFT:** Initial state when created by a Teacher or Administrator. Editable by author; unsubmitted and invisible to students.
2. **PENDING_APPROVAL:** Submitted for verification. Placed in the HoD or Administrator approval queue. Locked against author edits.
3. **APPROVED:** Verified by HoD, Admin, or Principal. Placed in the active broadcast dispatch queue or published to the notice feed.
4. **REJECTED:** Declined during review. Returned to author with mandatory feedback notes detailing required revisions.
5. **SCHEDULED:** Approved notice configured with a future publication timestamp (`scheduled\_at`). Automatically released by background cron workers.
6. **ARCHIVED:** Retired notice past its active validity period. Preserved in immutable database storage for historical audit trailing.

4.2 Priority Escalation Matrix

LOW Priority: General informational updates (e.g., club activities, voluntary workshops). Displayed in standard feed without intrusive alerts.

NORMAL Priority: Standard academic notifications (e.g., timetable updates, assignment submission deadlines). Displayed with standard category badges.

HIGH Priority: Crucial exam updates, fee payment deadlines. Highlighted banner placement, elevated feed ranking, and standard push notification.

EMERGENCY Priority: Immediate campus safety alerts, unexpected closures, severe weather warnings. Triggers high-priority push notifications, full-screen alert dialogs on active mobile devices, and automated campus public address speaker queue insertion.

4.3 Real-Time WebSocket Protocol Payload Specification

```
Sample Real-Time WebSocket Event Payload (`ws://backend:8000/ws/announcements`):
{ "event": "ANNOUNCEMENT_APPROVED", "timestamp": "2026-08-04T08:50:00Z", "data": { "id": 1042, "title":
"Final Semester Examination Schedule Released", "priority": "HIGH", "category": "Examinations",
"department_id": 2, "created_by": "Dr. Ramesh (HoD CSE)", "summary": "Final exam schedule for 6th/8th
semester CSE is now active." } }
```

5. EchoSphere AI Subsystem & Prompt Engineering

The EchoSphere AI Subsystem operates as an independent FastAPI microservice on Port 8001. It integrates state-of-the-art Large Language Models (Google Gemini 1.5/3.6) and local NLP pipelines to assist notice creators and provide domain-bounded academic assistance to students.

5.1 Core AI Microservice Capabilities

AI Service Component	Source Script Path	NLP Method & Model	Operational Functionality
Notice Drafting & Expansion	`services/text_expander_service.py`	Google Gemini LLM / Prompt Engineering	Converts raw bullet points into formal, grammatically polished academic announcements.
Executive Summarizer	`services/summarizer_service.py`	Abstractive Summarization NLP	Generates single-sentence executive summaries ideal for push notification text payloads.
Spam & Content Classifier	`services/spam_service.py`	Fine-Tuned Text Classifier	Scans incoming draft notices for unauthorized commercial spam, inappropriate language, or policy violations.
Duplicate Notice Detector	`services/duplicate_service.py`	Sentence Vector Embedding Cosine Similarity	Computes semantic similarity against active database notices to prevent duplicate announcements.
Priority & Category Tagger	`services/priority_service.py`	Urgency Sentiment Analysis	Analyzes text urgency keywords to recommend appropriate priority tags ('HIGH', 'EMERGENCY') and category tags.
Context-Aware Student Q&A;	`services/gemini_service.py`	Domain-Bounded RAG Engine	Answers student questions regarding branch timetables, subject syllabi, and official notices with strict domain guardrails.

5.2 Gemini LLM Prompt Template Specifications

```
-- System Prompt Template for Notice Expansion (`services/text_expander_service.py`) -- SYSTEM_PROMPT =  
''' You are EchoSphere AI, an academic notice drafting assistant for engineering colleges. Your task is  
to take raw, informal bullet points provided by faculty members and convert them into a formal,  
grammatically polished academic announcement. Guidelines: 1. Maintain formal academic tone. 2. Do not  
hallucinate dates, venue names, or contact details not provided in the prompt. 3. Highlight key  
deadlines, target semesters, and action items clearly. 4. Output clean markdown format suitable for  
mobile feed rendering.''' -- System Prompt Template for Domain-Bounded Student Q&A;  
(`services/gemini_service.py`) -- STUDENT_QA_PROMPT = ''' You are EchoSphere Assistant, a strict college  
query assistant for students. You ONLY answer questions related to college timetables, exam notices,  
department affairs, and official notices. If the user asks an out-of-scope question (e.g. general  
programming, movies, sports), respond with: 'I am EchoSphere Academic Assistant. I can only assist you  
with college announcements, branch timetables, and academic notices.' '''
```

5.3 Strict Domain Guardrails & Misuse Prevention

AI DOMAIN GUARDRAILS & SAFETY POLICY

Unlike general-purpose conversational chatbots, EchoSphere AI is explicitly prompt-engineered and domain-restricted. If a student asks out-of-scope questions (e.g., general entertainment, coding help, non-academic queries), the assistant gracefully declines and refocuses the conversation strictly on college announcements, academic timetables, and departmental affairs.

6. Frontend Architecture & Controller Code Signatures

The EchoSphere Frontend is developed using Flutter 3.x and Dart 3.x, targeting Android, Windows Desktop, Linux, and iOS. The codebase is structured under `frontend/lib/` using modular layer separation:

Frontend Directory Structure:
frontend/lib/
■■■■ ai/ # AI Assistant UI components & speech handling
■■■■ constants/ # Global colors, typography, API route constants
■■■■ controllers/ # Reactive State Controllers (Auth, Announcement, AI, Notification, Theme)

```
■■■■ models/ # Strong-typed Dart data models (User, Announcement, Category, UI State)
■■■■ screens/ # Modular Feature Screens
■ ■■■■ admin/ # UserManagementPage, AnnouncementManagementPage
■ ■■■■ announcements/ # AnnouncementDetailPage, ApprovalQueuePage, SpeakerQueuePage, ArchivePage
■ ■■■■ auth/ # LoginScreen with dynamic role tab selectors
■ ■■■■ home/ # HomePage with role-adaptive feed and drawer
■ ■■■■ notifications/ # NotificationCenterPage
■ ■■■■ profile/ # ProfilePage & Account Settings
■■■■ services/ # EchoSphereApiService, WebSocketListener, Soundpool Audio
■■■■ utils/ # Helpers, Date formatters, Validation Regex
■■■■ widgets/ # Reusable Card, Chip, Dialog & Banner widgets
```

6.1 Core Controller Method Signatures (Dart)

```
// AuthController (`lib/controllers/auth_controller.dart`) class AuthController extends ChangeNotifier {
User? _currentUser; String? _jwtToken; bool _isLoading = false; Future login({ required String
identifier, required String password, String? employeeId, }) async { ... } bool
isRegisteredStaffEmployeeId(String empId) { final cleanId = empId.trim().toUpperCase(); const
registeredStaffIds = ['DBITADM001', 'ADM001', 'DEVADM01', 'PRI001', 'HOD001', 'TCH001']; return
registeredStaffIds.contains(cleanId) || cleanId.startsWith('DBIT'); } } // AnnouncementController
(`lib/controllers/announcement_controller.dart`) class AnnouncementController extends ChangeNotifier {
List _announcements = []; List _approvalQueue = []; Future fetchAnnouncements({String? status, String?
category}) async { ... } Future createAnnouncement(AnnouncementCreateRequest request) async { ... }
Future approveAnnouncement(int id, {String? notes}) async { ... } Future rejectAnnouncement(int id,
{required String reason}) async { ... } }
```

6.2 Zero Layout Overflow Technical Constraint

Zero Overflow Technical Implementation: Mobile devices present diverse display aspect ratios (e.g., 320px compact mobile screens to 12.9-inch tablets and 4K desktop windows). To guarantee zero pixel layout overflow errors (such as Flutter's Yellow-and-Black RenderFlex overflow stripes), all components enforce dynamic constraints:

- Form fields, stat headers, and title banners are wrapped inside SingleChildScrollView and ListView.builder.
- Flexible horizontal action bars use Wrap with explicit spacing and runSpacing parameters.
- Text elements inside row layouts use Expanded or Flexible with TextOverflow.ellipsis to prevent truncation line overflow.

6.3 Comprehensive Screen-by-Screen Specification

Screen / Dialog	Target Route	Role Accessibility	Key Features & Interactivity
LoginScreen	`/login`	Unauthenticated / All	Role-selector tabs, animated logo, dynamic input fields switching between USN, Employee ID, and Email. Password visibility toggle.
HomePage	`/home`	All Authenticated Roles	Role-adaptive navigation drawer, search bar, category chips, pull-to-refresh feed, floating AI assistant button, unread notification badge.
AnnouncementDetailPage	`/announcements/:id`	All Authenticated Roles	Full notice content, emergency alert banner, priority tag badge, author profile metadata, text-to-speech audio reader, file attachments.
ApprovalQueuePage	`/approvals`	HoD, Admin, Principal	Pending review queue cards, batch review tools, approve with optional notes, reject with mandatory feedback reason modal.
UserManagementPage	`/admin/users`	College Admin, HoD, DevAdmin	Data table of registered staff/students, create user dialog, role & department assignment dropdowns, force password reset trigger.
SpeakerQueuePage	`/speaker-queue`	HoD, Admin, Principal	Physical campus public address speaker broadcast queue manager, priority re-ordering, audio synthesis preview playback.

7. Complete REST API Endpoint Specification & Schemas

The Core Backend exposes RESTful endpoints structured under `/api/v1/`. All requests must include standard HTTP headers (`Content-Type: application/json`, `Authorization: Bearer `).

HTTP Method	Endpoint Route	Role Guard	Request Body / Params	Response Model & Function
POST	/api/v1/auth/login`	Public	`identifier`, `password`, `employee_id`	`TokenResponse`: Returns signed JWT, user role, USN/EmplID, department ID.
POST	/api/v1/announcements`	Staff/HoD/Admin	`title`, `description`, `priority`, `category_id`	`AnnouncementResponse`: Creates draft notice and returns generated record ID.
GET	/api/v1/announcements/`	All Roles	Query params: `status`, `category`, `search`	`List[AnnouncementResponse]`: Retrieves role-filtered announcement feed.
POST	/api/v1/announcements/{id}/submit`	Teacher, HoD	`announcement_id` (Path)	`AnnouncementResponse`: Transitions state from `DRAFT` to `PENDING_APPROVAL`.
POST	/api/v1/announcements/{id}/approve`	HoD, Admin, Principal	`comments` (Optional text)	`AnnouncementResponse`: Approves notice, triggers WebSockets & push dispatch.
POST	/api/v1/announcements/{id}/reject`	HoD, Admin, Principal	`comments` (Mandatory text reason)	`AnnouncementResponse`: Rejects notice and returns to author with feedback.
POST	/api/v1/ai/expand`	Staff/Admin	`text` (Raw bullet points)	`JSON`: Returns expanded, formal academic announcement text.
POST	/api/v1/ai/summarize`	All Roles	`text` (Full notice text)	`JSON`: Returns single-sentence executive summary payload.

7.1 OpenAPI Request & Response JSON Schema Specifications

```
// POST /api/v1/auth/login Request Body: { "identifier": "1EC22CS001", "password": "StudentPassword123!",
"employee_id": null } // POST /api/v1/auth/login Response Body (TokenResponse): { "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "token_type": "bearer", "role": "Student", "user_id": 402,
"full_name": "Rahul Sharma", "official_email": "rahul.1ec22cs001@college.edu", "usn": "1EC22CS001",
"employee_id": null, "department_id": 2 }
```

8. Backend Database Schemas & PostgreSQL DDL Scripts

EchoSphere utilizes PostgreSQL 15 as its relational database management system. Models are declared in backend/app/models/ using SQLAlchemy 2.0 ORM base classes.

```
-- Core PostgreSQL Schema DDL Script for EchoSphere v2.2 --
CREATE TABLE roles ( id SERIAL PRIMARY KEY,
name VARCHAR(50) UNIQUE NOT NULL, description TEXT );
CREATE TABLE departments ( id SERIAL PRIMARY KEY,
code VARCHAR(20) UNIQUE NOT NULL, name VARCHAR(150) NOT NULL );
CREATE TABLE users ( id SERIAL PRIMARY KEY,
full_name VARCHAR(150) NOT NULL, username VARCHAR(50) UNIQUE NOT NULL, official_email VARCHAR(150)
UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, employee_id VARCHAR(30) UNIQUE, usn VARCHAR(30)
UNIQUE, semester INT, section VARCHAR(10), role_id INT REFERENCES roles(id), department_id INT REFERENCES
departments(id), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
CREATE INDEX ix_users_username ON users(username);
CREATE INDEX ix_users_official_email ON users(official_email);
CREATE TABLE announcements ( id SERIAL PRIMARY KEY,
title VARCHAR(255) NOT NULL, description TEXT NOT NULL, status
VARCHAR(30) DEFAULT 'DRAFT' NOT NULL, priority VARCHAR(30) DEFAULT 'NORMAL' NOT NULL, emergency_level
VARCHAR(30) DEFAULT 'NORMAL' NOT NULL, scheduled_at TIMESTAMP, created_by INT REFERENCES users(id) NOT
NULL, category_id INT REFERENCES announcement_categories(id) NOT NULL, created_at TIMESTAMP DEFAULT
```

```
CURRENT_TIMESTAMP ); CREATE INDEX ix_announcements_title ON announcements(title);
```

Table Name	Columns & Field Types	Constraints & Indexes	Foreign Keys & Relationships
users	<code>`id` (Int, PK)</code> <code>`username` (Varchar)</code> <code>`official_email` (Varchar)</code> <code>`password_hash` (Varchar)</code> <code>`employee_id` (Varchar)</code> <code>`usn` (Varchar)</code> <code>`semester` (Int)</code> <code>`section` (Varchar)</code> <code>`role_id` (Int)</code> <code>`department_id` (Int)</code>	PK on <code>`id`</code> Unique on <code>`username`</code> Unique on <code>`official_email`</code> Unique on <code>`employee_id`</code> Unique on <code>`usn`</code> Index on <code>`username`</code> , <code>`official_email`</code>	FK <code>`roles.id`</code> FK <code>`departments.id`</code> Has many <code>`Announcements`</code> , <code>`Approvals`</code> , <code>`Notifications`</code> , <code>`AuditLogs`</code> .
announcements	<code>`id` (Int, PK)</code> <code>`title` (Varchar)</code> <code>`description` (Text)</code> <code>`status` (Enum)</code> <code>`priority` (Enum)</code> <code>`emergency_level` (Enum)</code> <code>`scheduled_at` (DateTime)</code> <code>`created_by` (Int)</code> <code>`category_id` (Int)</code>	PK on <code>`id`</code> Index on <code>`title`</code> , <code>`status`</code> , <code>`priority`</code> Default <code>`status`=DRAFT`</code> Default <code>`priority`=NORMAL`</code>	FK <code>`users.id`</code> FK <code>`announcement_categories.id`</code> Has many <code>`Approvals`</code> , <code>`Deliveries`</code> .
announcement_approvals	<code>`id` (Int, PK)</code> <code>`announcement_id` (Int)</code> <code>`approver_id` (Int)</code> <code>`status` (Enum)</code> <code>`comments` (Text)</code> <code>`created_at` (Timestamp)</code>	PK on <code>`id`</code> Index on <code>`announcement_id`</code> Index on <code>`approver_id`</code>	FK <code>`announcements.id`</code> FK <code>`users.id`</code> Links notice to reviewing HoD / Admin / Principal.
audit_logs	<code>`id` (Int, PK)</code> <code>`user_id` (Int)</code> <code>`action` (Varchar)</code> <code>`target_resource` (Varchar)</code> <code>`ip_address` (Varchar)</code> <code>`created_at` (Timestamp)</code>	PK on <code>`id`</code> Index on <code>`user_id`</code> Index on <code>`created_at`</code> Immutable append-only	FK <code>`users.id`</code> Tracks login, creation, approval, and security events.
notifications	<code>`id` (Int, PK)</code> <code>`user_id` (Int)</code> <code>`announcement_id` (Int)</code> <code>`is_read` (Boolean)</code> <code>`delivered_at` (Timestamp)</code>	PK on <code>`id`</code> Default <code>`is_read`=False`</code> Index on <code>`user_id`</code> , <code>`is_read`</code>	FK <code>`users.id`</code> FK <code>`announcements.id`</code> Per-user read status tracker.
speaker_queue	<code>`id` (Int, PK)</code> <code>`announcement_id` (Int)</code> <code>`status` (Varchar)</code> <code>`broadcast_time` (Timestamp)</code>	PK on <code>`id`</code> Index on <code>`status`</code>	FK <code>`announcements.id`</code> Manages campus PA audio broadcast queue.

8.1 Immutable Audit Logging Subsystem

Administrative accountability is guaranteed by the ``AuditLog`` model. Whenever a user authenticates, creates an announcement, approves or rejects a notice, or triggers a password reset, an immutable audit entry is recorded containing user ID, operation action string, target resource ID, client IP address, and server timestamp. Audit logs are non-deletable.

9. DevOps, Deployment, Benchmarks & Future Roadmap

EchoSphere is designed for containerized deployment across cloud infrastructure or local institutional servers. Production environments utilize Docker Compose orchestrating the Core Backend API, AIML Engine, and PostgreSQL database.

9.1 Complete Production Docker Orchestration Config

```
docker-compose.yml:
version: '3.8'
services:
  backend:
    build: ./backend
    ports: ['8000:8000']
    environment:
      - DATABASE_URL=postgresql+asyncpg://postgres:secret@db:5432/echosphere
      - JWT_SECRET_KEY=production_super_secret_jwt_key_2026
      - AIML_SERVICE_URL=http://aiml:8001
    depends_on: [db]

  aiml:
    build: ./AIML
    ports: ['8001:8001']
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}

  db:
    image: postgres:15-alpine
    ports: ['5432:5432']
    environment:
      - POSTGRES_DB=echosphere
      - POSTGRES_PASSWORD=secret
    volumes: ['postgres_data:/var/lib/postgresql/data']
    volumes: { postgres_data: }
```

9.2 Database Migrations via Alembic

Database schema evolutions are versioned using Alembic. Schema updates are automatically executed during container deployment via:

alembic revision --autogenerate -m 'migration_name' and alembic upgrade head.

9.3 System Verification & Testing Suite Metrics

The Core Backend includes automated pytest test suites covering:

- **JWT Verification** (`test_jwt.py`): Validates token generation, cryptographic signing, expiration handling, and signature rejection.
- **Password Security** (`test_password.py`): Tests bcrypt hashing performance and password match verification.
- **API Integration Suite** (`test_api.py`): Tests full CRUD workflows for announcements, approval state transitions, and RBAC permission guards.

9.4 Future Enhancements & Strategic Roadmap

1. **iOS App Store Distribution:** Deploy sideloading repository (AltStore/SideStore) and official Apple App Store build.
2. **Web-Based Executive Dashboard:** React/Next.js administrative portal for institutional analytics and broad management.
3. **Multilingual PA Speaker Synthesis:** AI text-to-speech voice synthesis in regional languages for physical campus public address speakers.
4. **Offline Mesh Synchronization:** Peer-to-peer bluetooth notice sharing for emergency situations during campus network outages.

MASTER SPECIFICATION VERIFICATION & APPROVAL

Master specification compiled and verified against EchoSphere v2.2 codebase (frontend/lib, backend/app, AIML). All pages fully validated.